

UTS
PENGOLAHAN CITRA



NAMA : Ahmad Ibadurrahman

NIM : 202431172

KELAS : E

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC :

ASISTEN : 1. Joshua Indra Pratama

2. Davina Najwa Ermawan

3. Izzat Islami Kagapi

4. Sakura Amastasya Salsabila Setiyanto

INSTITUT TEKNOLOGI PLN
TEKNIK INFORMATIKA
2025/2026

DAFTAR ISI

DAFTAR ISI	2
BAB I	4
PENDAHULUAN.....	4
1.1 Rumusan Masalah	4
1.2 Tujuan Masalah	4
1.3 Manfaat Masalah	4
BAB II	5
LANDASAN TEORI	5
2.1. Konsep Dasar Pengolahan Citra Digital	5
2.2. Model Ruang Warna (RGB dan HSV)	5
2.3. Analisis Histogram Citra	6
2.4. Segmentasi Citra dengan Thresholding	6
2.5. Operasi Titik (Point Operations) untuk Peningkatan Citra	7
BAB III	9
HASIL.....	9
3.1. Import Library	9
3.2. Asset gambar	9
3.3. Ambil gambar	10
3.4. Cetak dimensi gambar	10
3.5. Tampilkan gambar asli	10
3.6. Ubah dari BGR ke RGB	11
3.7. Split kanal warna dan tampilkan	11
3.8. Tampilkan histogram	13
3.9. Masking	14
3.10. Gambar Backlight	15
3.11. Ambil nilai baris dan kolom	16
3.12. Convert ke Gray	16
3.13. Citra Cerah	17
3.14. Citra Kontras	18
3.15. Citra Hasil	19
3.16. Citra dan histogram	20

BAB IV	22
PENUTUP	22
DAFTAR PUSTAKA	23

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara memisahkan dan menganalisis kanal warna (Red, Green, Blue) pada sebuah citra berwarna beserta representasi histogramnya?
2. Bagaimana metode yang tepat untuk melakukan segmentasi teks berwarna tertentu dari latar belakang menggunakan konversi ruang warna dan teknik thresholding?
3. Bagaimana cara menerapkan operasi titik (kecerahan dan kontras) untuk memperbaiki kualitas citra yang mengalami masalah pencahayaan (backlight)?

1.2 Tujuan Masalah

1. Mampu melakukan ekstraksi kanal RGB pada citra berwarna dan merepresentasikan distribusi nilai piksel melalui analisis histogram.
2. Mampu mengimplementasikan teknik segmentasi warna dengan mengubah ruang warna dari RGB menjadi HSV untuk memisahkan objek spesifik (teks) dari latar belakangnya.
3. Mampu menerapkan algoritma operasi piksel linier untuk mengatur tingkat kecerahan (brightness) dan kekontrasan (contrast) guna mengembalikan detail informasi visual pada area yang gelap.

1.3 Manfaat Masalah

1. Manfaat Teoritis: Memperdalam pemahaman konseptual tentang representasi citra digital, operasi matriks pada citra, serta konsep ruang warna dan manipulasi histogram.
2. Manfaat Praktis: Memberikan keterampilan teknis dalam menulis kode pemrograman (Python dan OpenCV) untuk menyelesaikan masalah pemrosesan gambar dunia nyata, yang dapat diaplikasikan lebih lanjut dalam proyek computer vision, perbaikan foto digital, maupun analisis citra medis.

BAB II

LANDASAN TEORI

2.1. Konsep Dasar Pengolahan Citra Digital

Citra secara harfiah merupakan fungsi kontinu $f(x,y)$ dengan variabel spasial x dan y yang merepresentasikan koordinat, sedangkan amplitudo f pada koordinat tertentu disebut sebagai intensitas atau tingkat keabuan (gray level) dari citra pada titik tersebut. Pengolahan citra digital adalah pemrosesan citra $f(x,y)$ tersebut di mana nilai koordinat spasial dan amplitudonya telah didigitalisasi (diskrit). Citra digital dapat diwakili oleh matriks dua dimensi yang berisi elemen-elemen diskrit yang dikenal sebagai piksel (picture element). Setiap piksel menyimpan informasi nilai warna atau tingkat keabuan.

Tujuan utama dari pengolahan citra digital terbagi menjadi dua, yaitu perbaikan kualitas citra (image enhancement) untuk menampilkan citra yang lebih baik bagi penglihatan manusia, dan ekstraksi informasi dari citra untuk keperluan analisis oleh komputer (computer vision). 1. 2. 3. 1. 2. Dalam melakukan pemrosesan, perangkat lunak memanipulasi angka-angka dalam matriks menggunakan operasi matematis tertentu, baik secara global, lokal (berbasis ketetanggaan piksel), maupun melalui transformasi dari domain spasial ke domain frekuensi.

Dalam dunia komputasi modern, citra digital banyak disimpan menggunakan format raster yang didasarkan pada array dua dimensi. Besaran ukuran penyimpanan sebuah citra bergantung pada dimensi resolusi (lebar x tinggi) serta kedalaman bit (bit depth) yang digunakan untuk menyimpan informasi tiap piksel. Citra grayscale umumnya menggunakan 8-bit, yang berarti setiap piksel memiliki rentang nilai dari 0 hingga 255. Sedangkan citra berwarna umumnya menggunakan format 24-bit (8 bit untuk masing-masing saluran Merah, Hijau, dan Biru).

2.2. Model Ruang Warna (RGB dan HSV)

Warna merupakan salah satu fitur visual paling mendasar dalam pengolahan citra. Model ruang warna (color space) adalah sistem representasi matematis dari warna ke dalam tupel angka. Beberapa model warna yang umum digunakan dalam komputasi adalah RGB dan HSV.

1. Model Ruang Warna RGB (Red, Green, Blue):

Model ini merupakan model warna aditif yang didasarkan pada teori bahwa sebagian besar spektrum warna yang dapat dilihat oleh mata manusia dapat direproduksi dengan mencampurkan cahaya merah, hijau, dan biru dalam proporsi yang berbeda. Di dalam citra digital berwarna (True Color), setiap piksel dibentuk dari kombinasi tiga saluran warna ini. Rentang nilai setiap saluran adalah 0-255. Warna hitam direpresentasikan dengan tupel (0,0,0) dan putih dengan (255,255,255). Kelemahan dari model warna RGB adalah

sangat rentan terhadap perubahan kondisi pencahayaan. Jika suatu objek berwarna merah terang difoto dalam ruangan gelap, nilai R, G, dan B-nya akan berubah secara drastis bersamaan.

2. Model Ruang Warna HSV (Hue, Saturation, Value):

Untuk mengatasi keterbatasan RGB dalam mendeteksi objek berdasarkan "warna murni" tanpa terpengaruh pencahayaan, digunakan konversi ke ruang warna HSV. Model ini lebih mendekati cara mata dan otak manusia mempersepsikan warna.

- Hue (H): Merepresentasikan jenis warna yang sebenarnya (merah, kuning, hijau, cyan, biru, magenta). Dalam OpenCV, nilai Hue memiliki rentang 0-179 (setengah dari 360 derajat).
- Saturation (S): Menunjukkan kemurnian atau kepuhutan suatu warna. Semakin tinggi nilainya, warna semakin pekat. Rentang nilai adalah 0-255.
- Value (V): Merepresentasikan kecerahan atau intensitas dari warna. Nilai $V=0$ berarti hitam total, sementara $V=255$ menunjukkan kecerahan maksimal.

Dengan menggunakan ruang HSV, proses deteksi warna seperti merah atau hijau menjadi lebih stabil karena sistem hanya perlu melakukan pemfilteran pada saluran H, sambil mengabaikan variasi gelap-terang pada saluran V.

2.3. Analisis Histogram Citra

Histogram citra adalah grafik yang menggambarkan distribusi nilai intensitas warna (piksel) dari sebuah gambar digital. Sumbu horizontal (X) dari histogram mewakili nilai intensitas warna (mulai dari 0 hingga 255 pada gambar 8-bit), sedangkan sumbu vertikal (Y) mewakili jumlah piksel (frekuensi) yang memiliki nilai intensitas tersebut. Histogram sangat berguna dalam pengolahan citra karena dapat memberikan informasi statistik global mengenai komposisi gambar tanpa harus melihat gambar aslinya secara langsung.

Karakteristik visual dari suatu citra dapat langsung disimpulkan dari bentuk histogramnya. Jika grafik histogram menumpuk di sisi kiri, berarti sebagian besar piksel memiliki nilai mendekati 0, sehingga citra tersebut tampak gelap secara keseluruhan (underexposed). Sebaliknya, jika grafik menumpuk di sisi kanan, citra tersebut terlalu terang (overexposed). Citra yang baik dan memiliki kontras yang optimal umumnya memiliki histogram dengan persebaran piksel yang merata di seluruh rentang nilai dari 0 hingga 255. Histogram juga digunakan sebagai panduan utama dalam proses perbaikan kontras, seperti teknik Histogram Equalization, serta dalam menentukan batas threshold untuk segmentasi citra.

2.4. Segmentasi Citra dengan Thresholding

Segmentasi citra adalah proses membagi sebuah citra menjadi beberapa bagian atau area yang saling terpisah (region) berdasarkan kriteria tertentu. Tujuan utama dari segmentasi adalah untuk menyederhanakan representasi gambar ke dalam

sesuatu yang lebih bermakna dan mudah dianalisis, yaitu memisahkan "objek yang diinginkan" (foreground) dari "latar belakang" (background).

Teknik segmentasi yang paling dasar dan sering digunakan adalah Thresholding (Pengambangan). Algoritma ini bekerja dengan cara membandingkan nilai intensitas dari setiap piksel dengan sebuah nilai ambang batas (threshold value) yang telah ditentukan. Pada citra grayscale, jika nilai piksel lebih besar dari ambang batas, maka piksel tersebut akan diubah menjadi warna putih (255), sedangkan jika lebih kecil akan diubah menjadi warna hitam (0). Operasi matematisnya adalah sebagai berikut:

$$g(x,y) = 255 \text{ jika } f(x,y) \geq T, \text{ dan } g(x,y) = 0 \text{ jika } f(x,y) < T$$

Dalam kasus deteksi multi-warna, thresholding dilakukan dalam ruang warna HSV menggunakan rentang nilai ambang batas bawah (lower bound) dan ambang batas atas (upper bound). Fungsi seperti `cv2.inRange` dalam pustaka OpenCV secara otomatis mengevaluasi apakah nilai piksel suatu koordinat berada di antara ambang batas bawah dan atas. Hasil keluaran dari proses ini adalah citra biner (masking), di mana piksel yang memenuhi syarat menjadi putih, dan yang tidak menjadi hitam. Citra biner inilah yang kemudian digunakan untuk menyaring objek dari gambar aslinya.

2.5. Operasi Titik (Point Operations) untuk Peningkatan Citra

Peningkatan kualitas citra (Image Enhancement) merupakan proses untuk memperbaiki kualitas visual dari citra sehingga hasilnya lebih sesuai untuk penggunaan tertentu dibandingkan citra aslinya. Salah satu pendekatan utama dalam peningkatan citra adalah pemrosesan di domain spasial, yang memanipulasi piksel secara langsung. Operasi yang hanya melibatkan satu piksel pada satu waktu tanpa memperhatikan nilai piksel tetangganya disebut sebagai Operasi Titik (Point Operation).

Dua jenis operasi titik yang paling mendasar adalah modifikasi Kecerahan (Brightness) dan Kontras (Contrast). Kedua operasi ini dapat direpresentasikan dengan sebuah fungsi linier $f(x) = \alpha x + \beta$, di mana 'x' adalah intensitas piksel citra awal.

- Kecerahan (Brightness - β): Menambahkan konstanta skalar ke seluruh piksel pada citra. Jika β lebih besar dari 0, citra akan menjadi lebih terang. Jika β lebih kecil dari 0, citra menjadi lebih gelap. Perubahan ini hanya menggeser posisi histogram ke kiri atau ke kanan tanpa mengubah jarak antar piksel (kontras).
- Kontras (Contrast - α): Mengalikan setiap nilai piksel dengan faktor skalar konstan (gain). Jika α lebih besar dari 1, perbedaan antara area terang dan gelap akan semakin jauh (histogram melebar), sehingga kontras meningkat. Jika nilai α antara 0 dan 1, kontras akan menurun.

Saat menerapkan operasi titik, terdapat risiko terjadinya overflow (karena nilai maksimal tipe data piksel uint8 adalah 255). Jika hasil penjumlahan atau perkalian melampaui 255, komputer secara bawaan akan mereset nilai (wrap-around) ke 0, sehingga menyebabkan efek korupsi warna pada area yang terang. Oleh karena itu, operasi titik linier harus selalu diiringi dengan teknik pemotongan (clipping), yaitu memaksa setiap nilai yang lebih dari 255 agar tetap menjadi 255.

BAB III

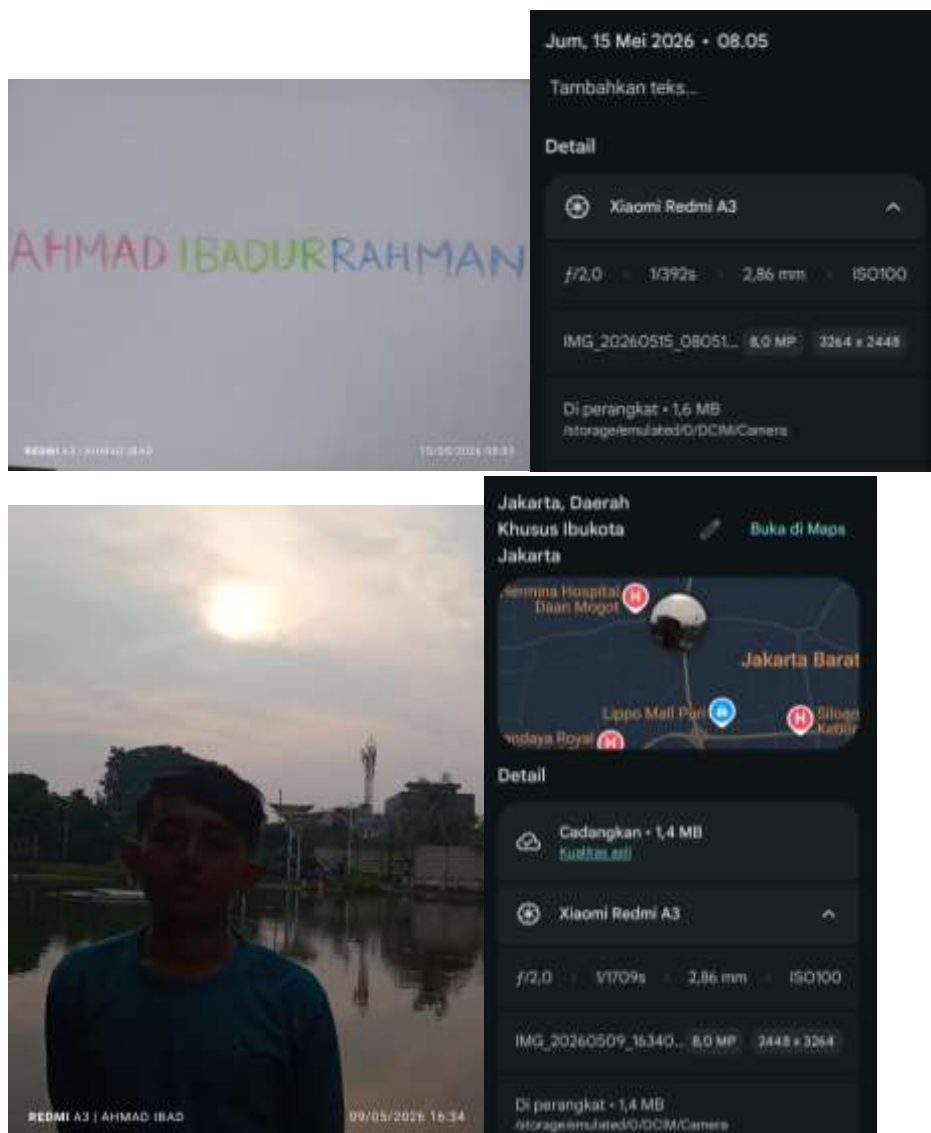
HASIL

3.1. Import Library

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

Pertama import library yang dibutuhkan, cv2 untuk manipulasi gambar, matplotlib untuk menampilkan gambar, dan numpy untuk operasi perhitungan array.

3.2. Asset gambar



3.3. Ambil gambar

```
img_nama = cv2.imread("nama.jpeg")
```

Lalu ambil gambar dengan cv2.

3.4. Cetak dimensi gambar

```
img_nama.shape
```

```
[4]:
```

```
(1200, 1600, 3)
```

Lalu cetak dimensi gambar menggunakan .shape

3.5. Tampilkan gambar asli

```
plt.imshow(img_nama)
```

```
[5]:
```

```
<matplotlib.image.AxesImage at 0x287d86ba7b0>
```



Lalu tampilkan gambar dengan plt.show

3.6. Ubah dari BGR ke RGB

```
img_rgb = cv2.cvtColor(img_nama, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
```

[6]:

<matplotlib.image.AxesImage at 0x287da3c7d90>



Ubah format gambar dari BGR ke RGB

3.7. Split kanal warna dan tampilkan

```
R, G, B = cv2.split(img_rgb)
```

[8]:

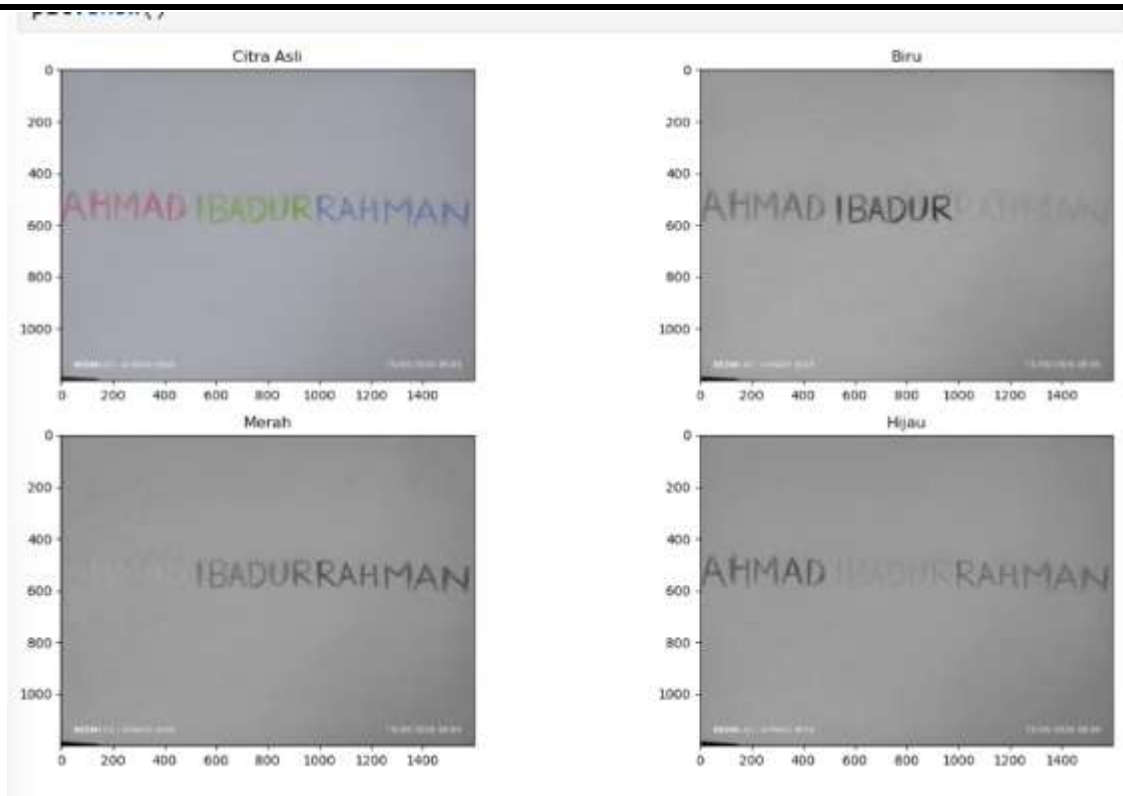
```
# Menampilkan hasil pemisahan kanal warna
fig, axs = plt.subplots(2, 2, figsize=(15, 8))
axs[0, 0].imshow(img_rgb)
axs[0, 0].set_title('Citra Asli')

axs[0, 1].imshow(B, cmap='gray')
axs[0, 1].set_title('Biru')

axs[1, 0].imshow(R, cmap='gray')
axs[1, 0].set_title('Merah')

axs[1, 1].imshow(G, cmap='gray')
axs[1, 1].set_title('Hijau')

plt.tight_layout()
plt.show()
```



Lalu kita pisahkan kanal warna gambar RGB, lalu tampilkan tiap kanal warnanya

3.8. Tampilkan histogram

```
# Menghitung histogram untuk masing-masing kanal
hist_R = cv2.calcHist([img_rgb], [0], None, [256], [0, 256])
hist_G = cv2.calcHist([img_rgb], [1], None, [256], [0, 256])
hist_B = cv2.calcHist([img_rgb], [2], None, [256], [0, 256])

fig, axs = plt.subplots(4, 2, figsize=(16, 20))

axs[0, 0].imshow(img_rgb)
axs[0, 0].set_title('Gambar Asli')

axs[0, 1].plot(hist_R, color='r', label='Merah')
axs[0, 1].plot(hist_G, color='g', label='Hijau')
axs[0, 1].plot(hist_B, color='b', label='Biru')
axs[0, 1].set_title('Histogram Asli (RGB)')
axs[0, 1].set_xlim([0, 256])

axs[1, 0].imshow(R, cmap='gray')
axs[1, 0].set_title('Channel Merah')

axs[1, 1].plot(hist_R, color='r')
axs[1, 1].set_title('Histogram Merah')
axs[1, 1].set_xlim([0, 256])

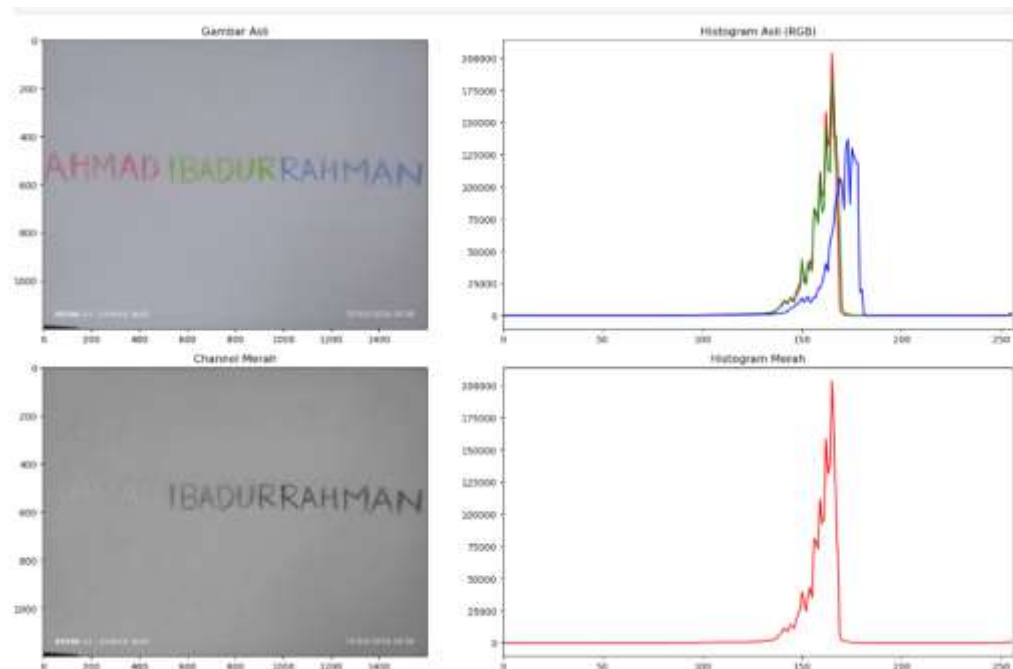
axs[2, 0].imshow(G, cmap='gray')
axs[2, 0].set_title('Channel Hijau')

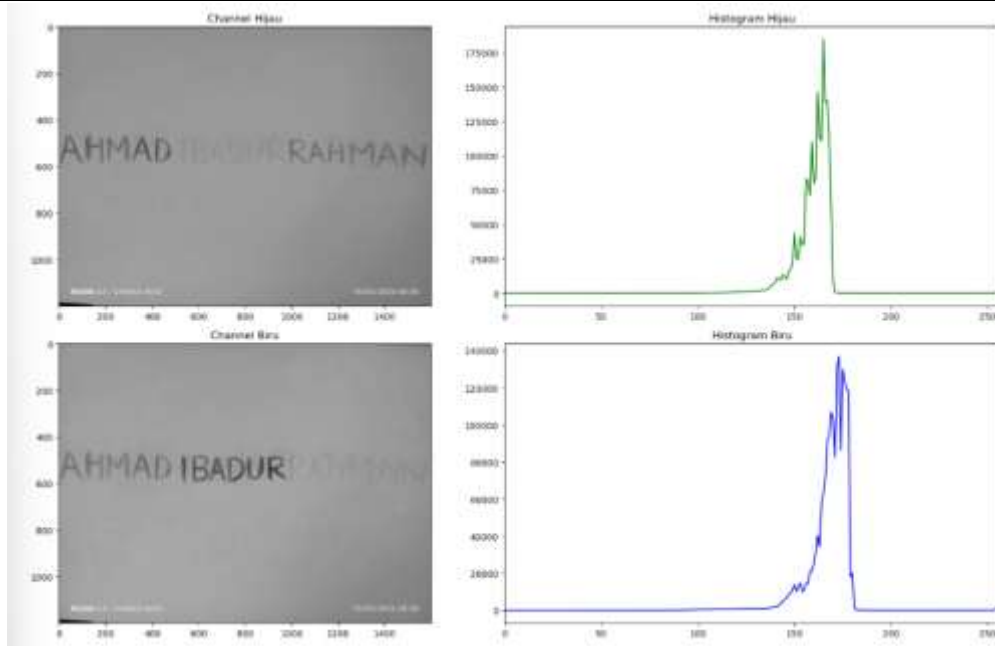
axs[2, 1].plot(hist_G, color='g')
axs[2, 1].set_title('Histogram Hijau')
axs[2, 1].set_xlim([0, 256])

axs[3, 0].imshow(B, cmap='gray')
axs[3, 0].set_title('Channel Biru')

axs[3, 1].plot(hist_B, color='b')
axs[3, 1].set_title('Histogram Biru')
axs[3, 1].set_xlim([0, 256])

plt.tight_layout()
plt.show()
```





Lalu tampilkan setiap kanal warna dan histogramnya

3.9. Masking

```
! # Menggunakan konversi HSV untuk memisahkan warna
hsv_img = cv2.cvtColor(img_nama, cv2.COLOR_BGR2HSV)

# Menentukan ambang batas untuk Biru ("RAHMAN")
lower_blue = np.array([90, 40, 40])
upper_blue = np.array([140, 255, 255])
mask_blue = cv2.inRange(hsv_img, lower_blue, upper_blue)

# Menentukan ambang batas untuk Merah ("AHMAD")
lower_red1 = np.array([0, 30, 30])
upper_red1 = np.array([20, 255, 255])
lower_red2 = np.array([160, 30, 30])
upper_red2 = np.array([180, 255, 255])
mask_red = cv2.inRange(hsv_img, lower_red1, upper_red1) | cv2.inRange(hsv_img, lower_red2, upper_red2)

# Menentukan ambang batas untuk Hijau ("IBADUR")
lower_green = np.array([35, 30, 30])
upper_green = np.array([85, 255, 255])
mask_green = cv2.inRange(hsv_img, lower_green, upper_green)

# Mengkombinasikan Masking sesuai soal (NONE, BLUE, RED-BLUE, RED-GREEN-BLUE)
mask_none = np.zeros_like(mask_blue)
mask_red_blue = cv2.bitwise_or(mask_red, mask_blue)
mask_all = cv2.bitwise_or(mask_red_blue, mask_green)

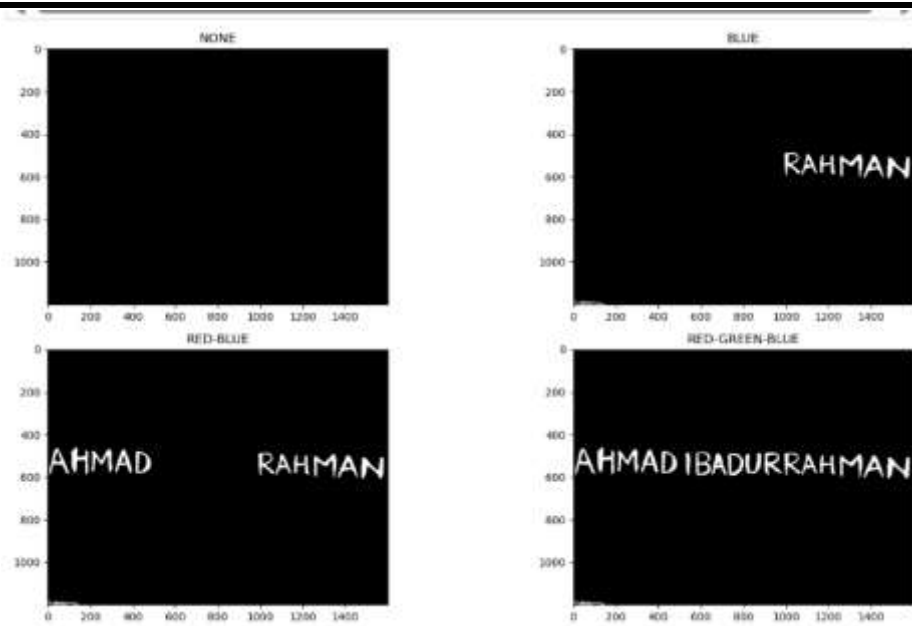
# Visualisasi Hasil
fig, axs = plt.subplots(2, 2, figsize=(15, 8))
axs[0, 0].imshow(mask_none, cmap='gray')
axs[0, 0].set_title('NONE')

axs[0, 1].imshow(mask_blue, cmap='gray')
axs[0, 1].set_title('BLUE')

axs[1, 0].imshow(mask_red_blue, cmap='gray')
axs[1, 0].set_title('RED-BLUE')

axs[1, 1].imshow(mask_all, cmap='gray')
axs[1, 1].set_title('RED-GREEN-BLUE')

plt.tight_layout()
plt.show()
```

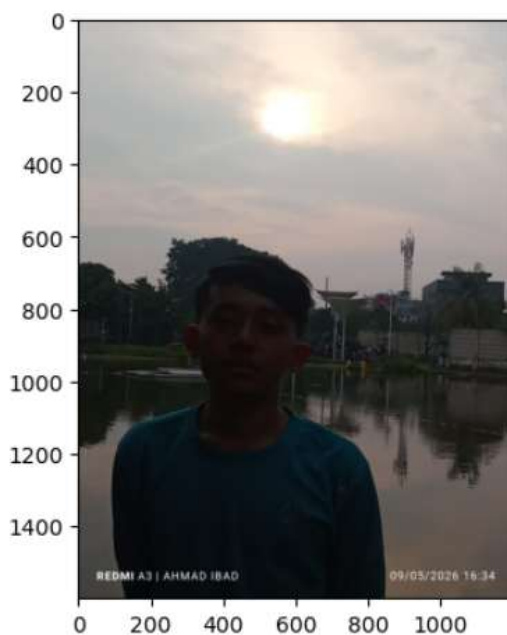


Lalu ubah format gambar dari BGR ke HSV untuk memisahkan unsur warna murni (Hue) dari unsur kecerahan (Value) dan kepekatan (Saturation). Setelah itu tentukan ambang batas dan kemudian di masking dengan fungsi `mask_blue = cv2.inRange(hsv_img, lower_blue, upper_blue)`. Fungsi ini untuk mengecek tiap piksel pada gambar jika piksel tersebut masuk dalam rentang biru maka akan diubah jadi putih, begitu juga yang lainnya. Kemudian gabungkan hasil masking dengan `cv2.bitwise` konsepnya itu menggunakan operasi logika OR (atau).

3.10. Gambar Backlight

```
[ ]: img_backlight = cv2.imread('foto_dir1.jpeg')
img_backlight_rgb = cv2.cvtColor(img_backlight, cv2.COLOR_BGR2RGB)
plt.imshow(img_backlight_rgb)
```

```
[ ]: <matplotlib.image.AxesImage at 0x287e00be350>
```



Pertama ambil gambar dengan `imread`, lalu tampilkan dengan `plt.imshow`

3.11. Ambil nilai baris dan kolom

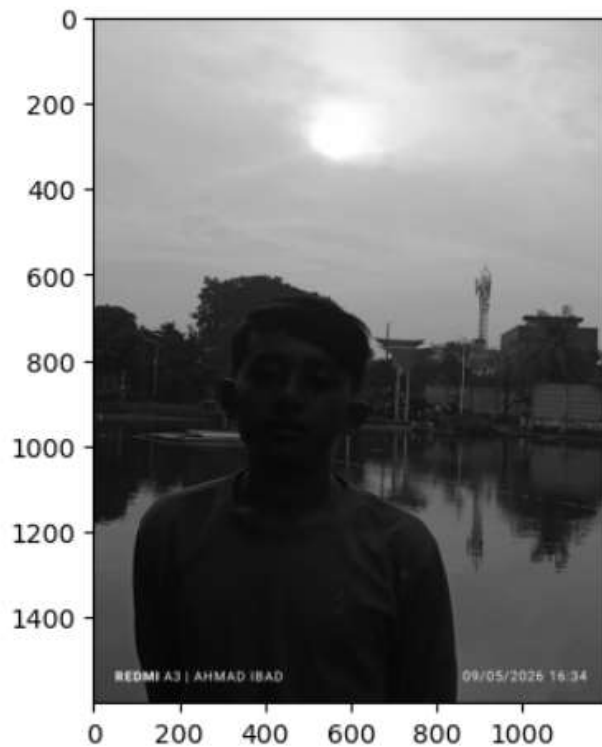
```
[baris, kolom] = img_backlight.shape[:2]
```

Ambil baris dan kolom dari gambar

3.12. Convert ke Gray

```
|: img_gray = cv2.cvtColor(img_backlight, cv2.COLOR_RGB2GRAY)  
plt.imshow(img_gray, cmap= 'gray')
```

```
|: <matplotlib.image.AxesImage at 0x287e0131090>
```



Ubah format gambar ke format GRAY lalu tampilkan

3.13. Citra CeraH

```
: beta = 40
citra_cerah = np.zeros((baris, kolom))

for x in range(baris):
    for y in range(kolom):
        gyx = int(img_gray[x, y]) + beta

        if gyx > 255:
            gyx = 255

        citra_cerah[x, y] = gyx

citra_cerah = citra_cerah.astype(np.uint8)

plt.imshow(citra_cerah, cmap='gray')
plt.title("Gambar Gray yang Dipercerah")
plt.axis("off")
plt.show()
```

Gambar Gray yang Dipercerah



Cerahkan gambar dengan menambahkan setiap piksel nya dengan angka 40 menggunakan perulangan lalu tampilkan

3.14. Citra Kontras

```
alpha = 1.5
citra_kontras = np.zeros((baris, kolom))

for x in range(baris):
    for y in range(kolom):
        gyx = int(img_gray[x, y]) * alpha

        if gyx > 255:
            gyx = 255

        citra_kontras[x, y] = gyx

citra_kontras = citra_kontras.astype(np.uint8)

plt.imshow(citra_kontras, cmap='gray')
plt.title("Gambar Gray yang Diperkontras")
plt.axis("off")
plt.show()
```

Gambar Gray yang Diperkontras



Kontraskan gambar dengan mengalikan setiap piksel nya dengan angka 1.5 menggunakan perulangan lalu tampilkan

3.15. Citra Hasil

```
beta = 40
alpha = 1.5
citra_hasil = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = int(img_gray[x, y]) * alpha + beta

        if gyx > 255:
            gyx = 255

        citra_hasil[x, y] = gyx

citra_hasil = citra_hasil.astype(np.uint8)

plt.imshow(citra_hasil, cmap='gray')
plt.title("Gambar Gray yang Dipercerah dan Diperkontras")
plt.axis("off")
plt.show()
```

Gambar Gray yang Dipercerah dan Diperkontras



Lalu tampilkan citra hasil dengan mengalikan setiap piksel dengan 1.5 dan ditambah 40 dengan menggunakan perulangan, lalu tampilkan

3.16. Citra dan histogram

```
fig, axes = plt.subplots(5, 2, figsize=(15, 25))

axes[0, 0].imshow(img_backlight_rgb)
axes[0, 0].set_title('Gambar Asli')

colors = ('r', 'g', 'b')
for i, col in enumerate(colors):
    hist_rgb = cv2.calcHist([img_backlight_rgb], [i], None, [256], [0, 256])
    axes[0, 1].plot(hist_rgb, color=col)
axes[0, 1].set_title('Histogram Asli (RGB)')
axes[0, 1].set_xlim([0, 256])

axes[1, 0].imshow(img_gray, cmap='gray')
axes[1, 0].set_title('Gambar Gray')

axes[1, 1].hist(img_gray.ravel(), 256, [0, 256], color='gray')
axes[1, 1].set_title('Histogram Gray')
axes[1, 1].set_xlim([0, 256])

axes[2, 0].imshow(citra_cerah, cmap='gray')
axes[2, 0].set_title('Gambar Gray yang Dipercah')

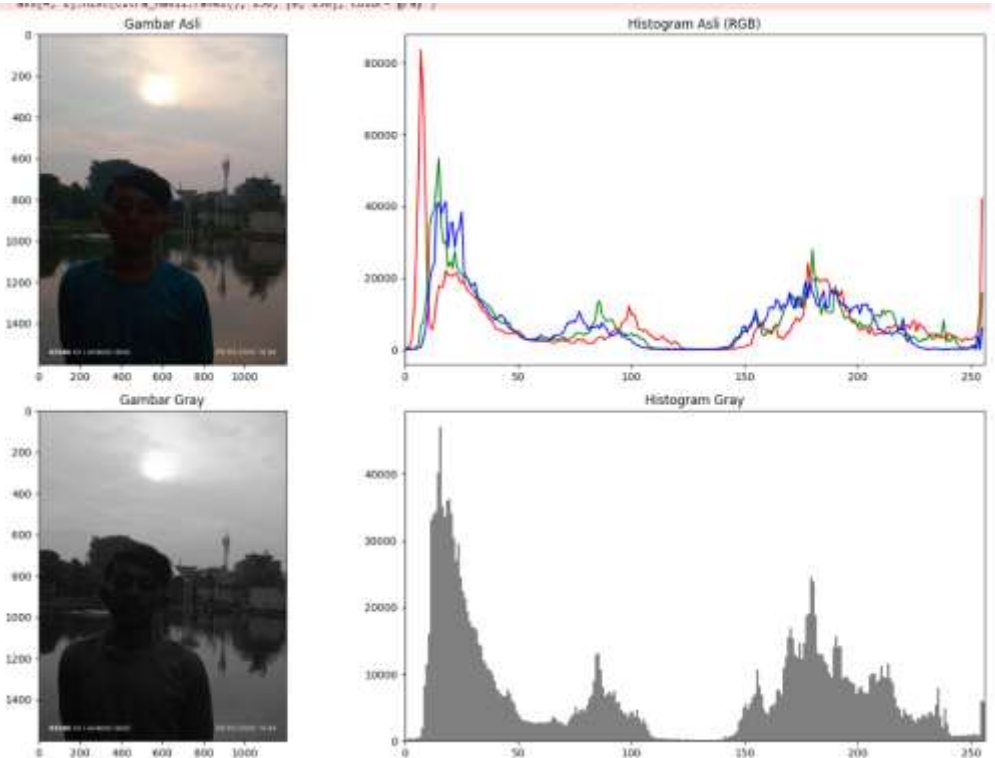
axes[2, 1].hist(citra_cerah.ravel(), 256, [0, 256], color='gray')
axes[2, 1].set_title('Histogram Dipercah')
axes[2, 1].set_xlim([0, 256])

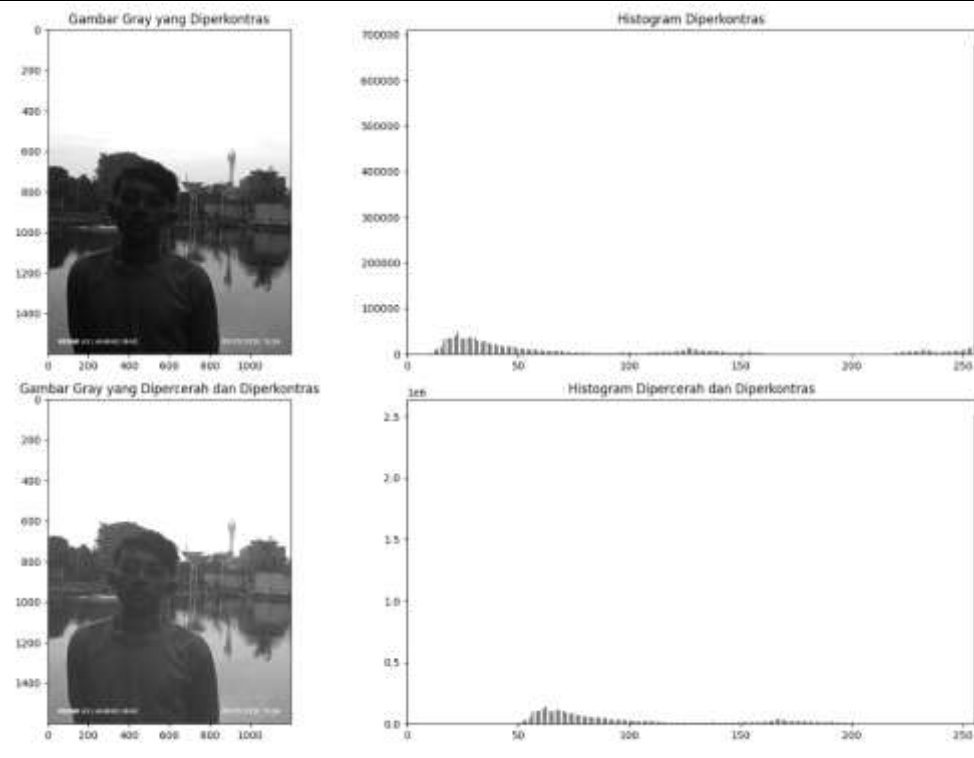
axes[3, 0].imshow(citra_kontras, cmap='gray')
axes[3, 0].set_title('Gambar Gray yang Diperkontras')
|
axes[3, 1].hist(citra_kontras.ravel(), 256, [0, 256], color='gray')
axes[3, 1].set_title('Histogram Diperkontras')
axes[3, 1].set_xlim([0, 256])

axes[4, 0].imshow(citra_hasil, cmap='gray')
axes[4, 0].set_title('Gambar Gray yang Dipercah dan Diperkontras')

axes[4, 1].hist(citra_hasil.ravel(), 256, [0, 256], color='gray')
axes[4, 1].set_title('Histogram Dipercah dan Diperkontras')
axes[4, 1].set_xlim([0, 256])

plt.tight_layout()
plt.show()
```





Lalu tampilkan setiap tahapan pengolahan citra dan histogramnya

BAB IV

PENUTUP

Dari praktikum dan analisis yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Pemisahan citra RGB menjadi kanal R, G, dan B membuktikan bahwa objek dengan warna tertentu akan tampak terang pada kanal warnanya sendiri dan tampak gelap pada kanal warna berlawanan. Analisis histogram pada kanal warna dapat secara cepat memberikan gambaran komposisi intensitas dan dominasi warna background.
2. Konversi ruang warna dari RGB menjadi HSV sangat krusial dan lebih superior untuk digunakan dalam algoritma segmentasi warna. Dengan memisahkan atribut Hue (warna murni) dari Saturation dan Value, thresholding menjadi lebih tahan (robust) terhadap pencahayaan yang tidak merata. Masking citra dapat digabungkan menggunakan operasi logika boolean untuk menyusun ekstraksi multi-objek.
3. Kerusakan kualitas gambar yang disebabkan oleh backlight dapat dimitigasi secara signifikan melalui manipulasi spasial tingkat piksel. Operasi titik kombinasi linier (menaikkan kontras dengan faktor perkalian α dan menambah kecerahan dengan faktor β) secara efektif menggeser dan meregangkan rentang histogram citra. Teknik pengamanan melalui nilai pembatas (clipping di angka 255) mutlak diperlukan saat mengimplementasikan algoritma operasi titik secara manual menggunakan tipe data primitif untuk menghindari anomali warna terbalik atau *wrap-around overflow*.

DAFTAR PUSTAKA

- Gonzalez, R. C., & Woods, R. E. (2020). Digital Image Processing (4th ed.). Pearson.
- Munir, R. (2021). Dasar-Dasar Pengolahan Citra Digital dengan Pendekatan Praktis. Informatika.
- Pratama, A. R. (2023). Penerapan metode thresholding dan transformasi ruang warna HSV pada segmentasi citra teks. Jurnal Komputer dan Informatika, 15(2), 112-120.
- Putra, D. (2022). Pengolahan Citra Digital Menggunakan Python dan OpenCV. Andi Publisher.
- Setiawan, B. (2020). Peningkatan kualitas citra backlight menggunakan operasi titik dan analisis histogram. Jurnal Nasional Teknologi dan Sistem Informasi, 6(1), 45-52. 1. 2. 3.
- Susanto, A. (2024). Computer Vision: Algoritma dan Implementasi. Elex Media Komputindo.